

and Bangalore. By examining each of the documents in our collection, we can build a term document incidence matrix of size  $(t \times N)$ , where each row represents a particular term and each column represents a document in collection. The matrix entry  $M[i][j]$  is 1 if the term  $t_i$  appears in document  $d_j$ . Else, it is 0. For our simple example, if  $N = 10^6$  and  $k = 10^3$ , the size of the matrix is  $10^9$ . In reality, this matrix can be very huge and, hence, efficient ways of representing the term document incidence matrix are needed.

A key point to note is that not all documents contain all terms. Therefore, many of the matrix entries are zero or, in other words, the matrix is sparse. Hence, we need an efficient representation of the sparse matrix. The standard data structure used for representing the term document incidence information is an 'inverted index'. The inverted index consists of a dictionary that contains all terms and a list that records for each term. The list that records the occurrence of a term in a document in our collection and also the position in the document in which it appears, is known as the 'postings list' or 'pos list', in short. The dictionary and the 'pos list' constitute the inverted index data structure for our document collection. Given the set of query terms for our query, we can retrieve the pos list for each term in the query. And by doing an intersection of the pos lists, we can get the list of documents in which all the four terms appear.

Given that each 'pos list' can be maintained as a linked list/array of tuples, with each tuple containing the document ID and the position of term in the document, how can we efficiently compute the intersection of the pos lists? Assume that each posting list is sorted in terms of the document ID and the dictionary is alphabetically sorted on the basis of 'terms'. We can use a modified version of 'sorted lists merge' to perform the intersection. I leave it as an exercise to the reader to come up with the actual algorithm.

We had glossed over the question of how we build the vocabulary for our document collection. This requires 'tokenisation', which is essentially the process of breaking the document into tokens. This is not a simple process, given that documents can be in different formats, and the structure and delineation of a document may not be very well defined. Tokenisation requires that the document is in a form that can be treated as a sequence of characters, which can be tokenised and parsed to yield the terms of the document. For instance, if your document collection consists of MS Word documents, they are in a specific Microsoft document format and need to be decoded into a sequence of characters before they can be tokenised. Delineation of documents is the task of identifying the unit of text that needs to be

considered as an individual document. At first glance, it seems simple to consider each file as a document. But in certain cases, a file needs to be split into multiple documents such as in the case of a mail storage file, which stores multiple email messages. In this case, we would like to treat each mail message as a separate document. Given the wide variety of file formats available today and various use-cases of information retrieval, character sequence decoding and document delineation can be quite complex.

Given the character sequence and the defined document unit, tokenisation is the process of splitting the character sequence into tokens. For instance, given the phrase, "Four score and seven years ago, our fathers brought forth on this continent," the tokens can be 'four', 'score', 'and', 'seven', 'years', 'ago', 'our', 'fathers', 'brought', 'forth', 'on', 'this', and 'continent'. The tokens are then analysed and pruned to remove common terms such as 'the', 'and' and 'on', etc. Two important pieces of tokenisation are 'stemming', which is the process of chopping off the ends of the words to get to a common base form, and 'lemmatisation', which is a more formal process of removing the ending to return a word form available in the dictionary. We will explore tokenisation in more detail in the next column.

## My 'must-read book' for this month

This month's book suggestion comes from me, and is on the subject of information retrieval. The book is 'Information Retrieval: Implementing and Evaluating Search Engines' by Stefan Buettcher, Charles L. A. Clarke, Gordon V. Cormack. It provides a good overview of information retrieval in the context of Web search engines.

If you have a favourite programming book or article that you think is a must-read for every programmer, please do send me a note with the book's name, and a short write-up on why you think it is useful so I can mention it in the column. This would help many readers who want to improve their software skills.

If you have any favourite programming questions/software topics that you would like to discuss on this forum, please send them to me, along with your solutions and feedback, at [sandyasm\\_AT\\_yahoo\\_DOT\\_com](mailto:sandyasm_AT_yahoo_DOT_com). Till we meet again next month, happy programming! 

## By: Sandya Mannarswamy

The author is an expert in systems software and is currently working with Hewlett Packard India Ltd. Her interests include compilers, multi-core and storage systems. If you are preparing for systems software interviews, you may find it useful to visit Sandya's LinkedIn group 'Computer Science Interview Training India' at <http://www.linkedin.com/groups?home=&gid=2339182> &HYPERLINK "http://www.linkedin.com/groups?home=&gid=2339182" &HYPERLINK "http://www.linkedin.com/groups?home=&gid=2339182" &gid=2339182"gid=2339182